

DevOps Engineering Technical Exercise

KAD Global Agency | Confidential

Role:	DevOps Engineer
Technology:	Terraform 1.6+ , Helm 3.13+ , kubectl 1.28+ , kind 0.20+ , Docker 24+ (Docker Desktop, Colima, or Rancher Desktop all fine), Conftest 0.46+ (older versions don't support Rego v1 syntax), Checkov 3.x , tflint 0.50+ , GitHub Actions . The full toolchain is open-source and runs locally; no cloud account required. Minimum 8 GB RAM allocated to your Docker runtime (the local cluster runs ~20 pods).
Estimated Time:	6–12 hours, depending on your familiarity with this exact stack. Plan more time if you have not worked with kind + ArgoCD + ESO + Kyverno before.
Submission:	Zip including <code>.git/</code> + README + sample <code>make verify</code> output
Send to:	muhammed.khan@truxel.ai, monica@kad.agency

1. Objective

Provision a small, self-contained Kubernetes platform locally that mirrors a production GitOps pattern. The bundle ships an application chart and a verification harness; you write the **Terraform**, the **ArgoCD** wiring, the per-environment **Helm values**, the **Kyverno** admission policies, the **Conftest** policies, and the **GitHub Actions** workflows that hold it all together.

We are evaluating **Terraform module design and environment separation**, **GitOps discipline** (app-of-apps, sync policies, drift handling), **policy-as-code** (admission control + static policy on rendered manifests + Terraform plans), **CI pipeline composition**, **secrets handling without static credentials**, and **operational hygiene** (no secrets in Git, reproducible boots, documented trade-offs). We are *not* evaluating cloud-vendor proficiency, application code, or chart authoring — we ship the application chart so you can focus on the DevOps surface.

2. Scenario

You are the first DevOps engineer on a small platform team. Your job is to stand up the deployment substrate that every future application will run on. To keep iteration fast and cost-free, the entire substrate runs locally on a single laptop via **kind** (Kubernetes in Docker), driven by Terraform. Once it boots, a GitOps controller (ArgoCD) takes over and continuously syncs applications from this repository.

The system has four moving parts:

- **The local cluster** — a kind-based Kubernetes cluster, provisioned by Terraform.
- **The platform components** — ArgoCD, the External Secrets Operator (ESO) with a local backend, and Kyverno — installed via Helm releases declared in Terraform.

- **The sample application** — a shipped Helm chart wrapping the public image `traefik/whoami:v1.10.4`. You write per-environment values overlays and the ArgoCD Application that points at it.
- **The verification harness** — a shipped `make verify` pipeline that boots the cluster, asserts the deployment is healthy, and validates your policies.

3. Provided Components

The exercise ships with everything you need to develop and self-assess against:

```
devops-exercise-bundle/
├── devops-engineer-exercise.pdf      # this document
├── README.md                       # quick-start guide
├── Makefile                        # the top-level entry points
├── helm/
│   ├── charts/
│   │   └── sample-app/             # full chart, references
├── traefik/whoami:v1.10.4
├── verify/
│   ├── verify.sh                  # orchestrates the full pipeline
│   ├── policies/                  # Conftest .rego files we test you against
│   └── bad-examples/              # deliberately-violating manifests, see §6.4
├── terraform/                     # empty scaffold – you fill it in
├── argocd/                         # empty scaffold – you fill it in
└── .github/workflows/              # empty scaffold – you fill it in
```

3.1 The sample application (shipped)

You do not need to write application code or chart templates. `helm/charts/sample-app/` contains a complete Helm chart that wraps the public image `traefik/whoami:v1.10.4`. It exposes HTTP on port `80` and responds to any path with request metadata as text. Pull-from-Docker-Hub is required; no local registry needed.

You may add **values overlay files** next to the chart (e.g. `helm/charts/sample-app/values.local.yaml`), but do not modify the chart templates or the shipped `values.yaml`.

Naming requirements the harness depends on:

- ArgoCD Application for the local env must have `metadata.name: sample-app` (filename `argocd/apps/sample-app.yaml`). The release name flows from the Application name, so the rendered Service becomes `svc/sample-app` on port `80` — that's what the harness probes.
- For local-stage: `metadata.name: sample-app-stage`, deployed to namespace `sample-app-stage`.
- ArgoCD itself must be installed in the `argocd` namespace.
- Your overlay values must *keep* the chart's defaults for `resources.limits.{cpu,memory}`, `securityContext.runAsNonRoot: true`, and a pinned image tag — the shipped helm-render policy will reject the rendered output if any of these are missing.
- Place your overlay file at `helm/charts/sample-app/values.local.yaml` (alongside the shipped chart). The harness looks here first.
- Your ArgoCD Application for the sample app must include `syncOptions: ["CreateNamespace=true", "ServerSideApply=true"]` — the bad-examples manifests in

verify/bad-examples/ assume the sample-app namespace exists, and the harness step 10 fails with a misleading error if it doesn't.

3.2 Source repository — how ArgoCD reaches your code

ArgoCD pulls source from Git. For this exercise you publish your work to a **public Git repository** (e.g. GitHub or GitLab) and point your ArgoCD `Application` resources at it over HTTPS. The harness assumes the repo is publicly readable — no credentials are wired into ArgoCD. Private repos are out of scope; if you use one, step 6 will not pass without additional repo-credentials work we do not require or grade.

The `argocd/` directory in the bundle is an empty scaffold; you author every manifest in it. Use the literal placeholder `<YOUR_GIT_REPO_URL>` for the `repoURL` field while iterating, then swap it for your real URL when you push. Before `make verify`:

1. Push your work to a public repo at least once.
2. Replace every `<YOUR_GIT_REPO_URL>` occurrence anywhere under `argocd/` (covers `root.yaml`, `apps/*.yaml`, and `projects/*.yaml`) with your repo's HTTPS clone URL (e.g. `https://github.com/<you>/<repo>.git`). A one-liner that works: `find argocd -name '*.yaml' -exec sed -i.bak 's#<YOUR_GIT_REPO_URL>#https://github.com/<you>/<repo>.git#g' {} +`.
3. Commit and push the URL change. ArgoCD reads the committed manifests, so the placeholder substitution must be in the pushed tree.

The harness recursively greps `argocd/` for the literal placeholder before waiting for ArgoCD — if any occurrence remains, the root-Application check (step 6) fails fast with a clear message. ArgoCD then pulls from your URL, so unpushed changes are invisible to it.

Network requirement: the kind cluster's ArgoCD pods must reach `github.com` / `gitlab.com` (or wherever you host) over HTTPS. On corporate laptops behind an egress proxy this may need extra configuration; verify with `kubectl -n argocd exec deploy/argocd-repo-server -- curl -sI https://github.com` if step 6 hangs.

3.3 The verification harness (shipped)

After your stack is in place, run:

```
make verify
```

This script boots the cluster from your Terraform, waits for ArgoCD to converge, asserts the sample application is reachable, attempts to apply a deliberately-violating manifest to prove your Kyverno policies fire, runs Conftest against your rendered ArgoCD manifests and Terraform plan, and inspects your repo for the required workflow files, the validation blocks in your Terraform modules, and the AI-tools-used disclosure in your README. Each check prints `PASS` or `FAIL` and the script exits non-zero on any failure. Target: **all 16 checks pass**.

By default the harness runs `terraform destroy` after a successful run to clean up. While iterating, set `KEEP=1 make verify` to keep the cluster around so you can poke at it. Reaching a green run with `KEEP=1` still counts as a pass.

4. Terraform Requirements

4.1 Layout

Your Terraform must follow this layout. The pattern mirrors how a real platform team separates reusable building blocks from per-environment compositions:

```
terraform/
├─ modules/
│   ├── kind-cluster/           # creates a kind cluster, exposes kubeconfig
│   └─ argocd-app/             # creates an ArgoCD Application + AppProject
pair
├─ envs/
│   ├── local/                 # the primary environment
│   └─ local-stage/            # a second environment (different namespace +
values overlay)
└─ backend.hcl.example         # template for backend init values
```

4.2 Modules — required

- `modules/kind-cluster/` — provisions a kind cluster using the `tehcyx/kind` provider. Outputs at least `kubeconfig_path`, the cluster endpoint, and the cluster CA cert. Must accept inputs for cluster name and node count.
- `modules/argocd-app/` — creates an ArgoCD `Application` resource plus its `AppProject`. Accepts inputs for app name, source repo URL, target namespace, project name, sync-policy options. Must default to `automated.prune = true` and `automated.selfHeal = true`.

Each module must include `main.tf`, `variables.tf`, `outputs.tf`, and `versions.tf` with pinned provider versions.

CRD-at-plan-time gotcha (important): the official `hashicorp/kubernetes` provider's `kubernetes_manifest` resource validates manifests against the cluster's discovery API at *plan* time. This means a single-pass `terraform apply` will fail if you try to create CRD resources (ArgoCD `Application`, ESO `ClusterSecretStore`, Kyverno `ClusterPolicy`) in the same apply that installs their CRDs via `helm_release`.

Recommended path: use `gavinbunney/kubectl` provider's `kubectl_manifest` resource for all CRD-shaped objects — it does not validate at plan time. (You may also do a two-stage apply with `-target=helm_release.*` first, but that complicates the `make verify` single-shot flow.)

Parallelism note: the kind cluster's API server is a single control-plane container with modest resources. Installing four `helm_release`s (cert-manager, ArgoCD, ESO, Kyverno) in parallel can overwhelm it — you may see TLS handshake timeouts mid-apply. The shipped `make verify` runs `terraform apply -parallelism=3` for this reason. If you invoke `terraform apply` directly, pass the same flag or chain the helm releases with `depends_on`.

4.3 Variable validation

At least **two** variables across your modules must declare a `validation` block. Examples: cluster name must match a regex, node count must be between 1 and 5, namespace must not be `default`. You pick which two; the harness inspects the AST for at least two validation blocks.

4.4 Backend convention

The exercise uses the **local** Terraform backend (no cloud state). However, your `backend.tf` must be parametrised such that `terraform init -backend-config=backend.hcl` is the documented init command. The repo must commit `backend.hcl.example` as a template; `backend.hcl` itself must be `.gitignore`'d. This mirrors the convention real teams use when state is later moved to S3 / GCS.

Bootstrap convention: because `backend.hcl` and `terraform.tfvars` are gitignored, your `make setup` target must copy their `*.example` counterparts into place if they're absent. A fresh clone of your repo must be able to run `make verify` without manual file copying.

4.5 Environments

`envs/local/` and `envs/local-stage/` must both call your two modules. They must:

- Deploy the sample application to different namespaces (e.g. `sample-app` vs `sample-app-stage`)
- Use different Helm values overlay files (you write both: `values.local.yaml`, `values.local-stage.yaml`)
- Show that the same modules cleanly support both environments (parameterisation done right)
- **Re-export the kind-cluster module's `kubeconfig_path` output at the env level.** The shipped harness reads `terraform output -raw kubeconfig_path` from the env directory to locate the cluster. Without this re-export, every `kubectl` call in the harness silently falls back to `localhost:8080` with confusing "connection refused" errors. Pattern:

```
output "kubeconfig_path" {
  value = module.cluster.kubeconfig_path
}
```

4.6 Plan-as-policy (Conftest on Terraform plan)

The harness runs `terraform plan -out=tfplan` followed by `terraform show -json tfplan | conftest test --policy verify/policies/terraform/ -`. You must satisfy the shipped policies:

- No `helm_release` resource may use a `version` that is unpinned, empty, or wildcarded — must be an exact version string.
- The `kind_cluster` name must be DNS-1123 compatible (lowercase alphanumeric and hyphens, no spaces, ≤ 32 chars).

Read the exact policies in `verify/policies/terraform/*.rego` before you write your Terraform.

5. ArgoCD Requirements

5.1 Layout

```
argocd/
├─ root.yaml                # the master Application (app-of-apps root)
├─ projects/
```

```

├── root.yaml                # AppProject for the root app itself
├── sample-app.yaml         # AppProject scoping for the sample app
└── apps/
    ├── sample-app.yaml     # ArgoCD Application for the local env
    └── sample-app-stage.yaml # ArgoCD Application for the local-stage env

```

5.2 Pattern

- The root Application is applied **once manually** during bootstrap (the harness does this for you).
- The root Application then watches `argocd/apps/` — adding a YAML there must make ArgoCD discover and deploy the new app on the next sync, with no further manual action.
- Every Application in `argocd/apps/` must have `syncPolicy.automated.prune` and `syncPolicy.automated.selfHeal` set to `true`.
- Each Application must reference its scoped AppProject (not `project: default`).

Two valid paths: (a) keep the ArgoCD Application resources as standalone YAML files in `argocd/apps/`, applied by the root via Git-folder discovery, or (b) create them inside your Terraform via the `argocd-app` module. **Use the YAML-files approach for the apps** — that matches the shipped harness, which checks `argocd/apps/*.yaml` with Conftest. Your `argocd-app` Terraform module is exercised by your Terraform code (e.g. you could create the AppProjects via the module) but the actual app manifests live as YAML.

5.3 Conftest on ArgoCD manifests

The harness runs `conftest test --policy verify/policies/argocd/ argocd/apps/*.yaml`. The shipped policies require:

- Every Application sets `spec.syncPolicy.automated.prune = true`
- Every Application sets `spec.syncPolicy.automated.selfHeal = true`
- No Application uses `spec.project: default`
- Every Application sets `spec.destination.namespace` explicitly

6. Policy & Security Requirements

6.1 Kyverno (admission-time enforcement)

Install Kyverno (via Terraform `helm_release`) and add a small custom Helm chart at `helm/charts/kyverno-policies/` containing at least these two `ClusterPolicy` resources:

1. `require-resource-limits` — deny any Pod whose containers do not declare both `resources.limits.cpu` and `resources.limits.memory`.
2. `disallow-latest-tag` — deny any Pod whose container image uses tag `:latest` or has no tag at all.

Policies must be **enforced** (not audit-only). Apply to all namespaces except `kube-system` and `kyverno`.

Kyverno autogen + ArgoCD drift: by default Kyverno auto-generates `autogen-*` rules from your `ClusterPolicy` at runtime to cover Deployments, StatefulSets, etc. ArgoCD then sees the live `.spec.rules` as drifted from what's in Git, and the Kyverno-policies Application can stay `OutOfSync` forever. Avoid this by either (a) annotating each `ClusterPolicy` with `pod-policies.kyverno.io/autogen-controllers: "none"` and matching on `Pod` directly, or (b) configuring ArgoCD to ignore the autogen rules via `spec.ignoreDifferences`. The harness does not inspect this Application's sync state, but a noisy `OutOfSync` will count against you in the code review.

6.2 Deploy Kyverno policies via ArgoCD

The `kyverno-policies` chart you write must be deployed by an ArgoCD `Application`, just like `sample-app`. This is the test that you understand the GitOps pattern applies to your policies too.

6.3 Conftest (static policy on rendered manifests)

The shipped `verify/policies/` contains Rego policies organised by target:

- `verify/policies/terraform/` — runs against the JSON plan output
- `verify/policies/argocd/` — runs against your `argocd/apps/*.yaml`
- `verify/policies/helm/` — runs against the output of `helm template helm/charts/sample-app/ --values helm/charts/sample-app/values.local.yaml`

Read them; they document exactly what your output must satisfy. You may add your own policies under `verify/policies/extra/` — harmless, not required.

6.4 Bad-examples folder (proves your Kyverno policies actually fire)

The bundle ships `verify/bad-examples/` with three deliberately-violating manifests:

- `no-limits.yaml` — a Pod without resource limits
- `latest-tag.yaml` — a Pod using `image: nginx:latest`
- `both.yaml` — a Pod violating both rules at once

The harness applies each of these and asserts that **Kyverno rejects all three**. If any one is admitted, the corresponding check fails.

6.5 External Secrets Operator (ESO)

Install ESO via Terraform. Configure a `ClusterSecretStore` backed by the **local Kubernetes-Secrets provider** (i.e. ESO reads from another Secret in the cluster instead of from a cloud secret manager). This mirrors the cloud-secret pattern locally at zero cost.

Demonstrate it by creating one `ExternalSecret` resource in the `sample-app` namespace that produces a Kubernetes `Secret` readable by the `sample-app` pod. The harness asserts the synced Secret exists and has the expected key.

6.6 Image scanning (Trivy)

Run Trivy in CI (§7.3) against the pinned image. We do not require zero CRITICAL findings on the upstream image — we evaluate that you wired the scan, captured its output, and uploaded the report as a CI artefact.

7. CI/CD Requirements (GitHub Actions)

Three workflows are required under `.github/workflows/`. Each must run on pull request and on push to `main`.

7.1 `terraform-plan.yml`

- `terraform fmt -check -recursive` on `terraform/`
- `terraform validate` in each env
- `tflint` on each env (provider plugin auto-installed)
- `terraform plan -out=tfplan` in `envs/local/`
- `terraform show -json tfplan | conftest test --policy ../../../../verify/policies/terraform/ -`
- `checkov -d terraform/` with `--quiet --soft-fail-on LOW,MEDIUM` (only HIGH/CRITICAL fail the build)

7.2 `policy-test.yml`

- `conftest test --policy verify/policies/argocd/ argocd/apps/*.yaml`
- For each chart in `helm/charts/`: run `helm lint`, then `helm template` piped to `conftest` against `verify/policies/helm/`

7.3 `security-scan.yml`

- `trivy image traefik/whoami:v1.10.4 --severity HIGH,CRITICAL --format json`
- Upload the JSON output as a workflow artefact named `trivy-report`
- Do *not* fail the build on findings — the upstream image's CVEs are not the candidate's problem; we evaluate that you wired the scan, not the result

8. The `make verify` Contract

Your work is graded primarily by the output of `make verify` running locally on a clean clone. The shipped harness performs these steps in order:

1. **Pre-flight** — checks `docker`, `terraform`, `helm`, `kubectl`, `kind`, `conftest`, `checkov`, and `tflint` are on `PATH`
2. **Lint** — `terraform fmt -check`, `terraform validate`
3. **Plan** — `terraform plan -out=tfplan` in `envs/local/`
4. **Plan-as-policy** — `conftest` against the JSON plan
5. **Apply** — `terraform apply` — cluster + ArgoCD + ESO + Kyverno come up
6. **Wait-for-ArgoCD** — root Application reports `Synced` + `Healthy`
7. **Wait-for-sample-app** — sample-app Application reports `Synced` + `Healthy`
8. **HTTP probe** — `kubectl port-forward` + `curl` → sample-app responds `200 OK`
9. **External Secret check** — the `ExternalSecret` in the sample-app namespace has synced; the target `Secret` exists
10. **Kyverno admission** — apply each file in `verify/bad-examples/`; assert Kyverno rejects each
11. **Conftest on ArgoCD** — `argocd/apps/*.yaml`
12. **Conftest on Helm-rendered output** — `helm template` piped through `Conftest`
13. **Checkov** — on `terraform/` (HIGH/CRITICAL only)
14. **AI disclosure** — `grep` `README` for an "AI tools used" (or equivalent) heading
15. **Workflows present** — all three GitHub Actions files exist under `.github/workflows/`
16. **Validation blocks** — at least two `validation { ... }` blocks under `terraform/modules/`
17. **Teardown** — `terraform destroy` runs automatically on a passing run. Set `KEEP=1` to skip teardown while iterating

Final line on a passing run: `✓ 16/16 checks passed`. On any failure the script exits non-zero and lists the failing checks. **We expect all 16 to pass.**

9. Pipeline Entry Point

Provide a top-level `Makefile` with these targets:

```
make setup      # checks the required tools are on PATH; copies *.example files
                # (backend.hcl.example -> backend.hcl, terraform.tfvars.example
                # -> terraform.tfvars) so a fresh clone can run make verify
make plan       # terraform plan only
make apply      # terraform apply (full bring-up)
make verify     # the full 16-check pipeline above
make destroy    # terraform destroy
make clean      # destroy + remove temp files
```

`make setup` does *not* install missing tools — it reports what's missing and exits. Document the install commands in your `README`.

10. Technical Expectations

- Terraform 1.6+. Pin all provider versions in `versions.tf`.

- kind 0.20+, Kubernetes 1.28+, Helm 3.13+, kubectl 1.28+.
- No real cloud account. No published artefacts on your side — the sample image is pulled from public Docker Hub.
- **Secrets:** nothing committed in plaintext. Use the shipped ESO + ClusterSecretStore pattern (§6.5) for any app secret. Manual reviewers spot-check for leaks.
- **All cluster mutations flow through Terraform or ArgoCD.** The only direct `kubectl apply` the harness performs is on the shipped `verify/bad-examples/` manifests, to prove your Kyverno policies fire.
- The full `make verify` pipeline must pass on a clean clone of your repository.

11. Documentation (Required)

Include a `README.md` at the root, concisely covering:

- **How to run** — the `make setup` → `make verify` path
- **Architecture** — one or two paragraphs describing the modules, environments, and how ArgoCD picks up apps
- **Module interface** — documented inputs and outputs of `kind-cluster` and `argocd-app`
- **Secrets approach** — how ESO reads, what the `ClusterSecretStore` points at
- **Trade-offs** — what you cut for the time budget, what you would build next in a real production setting (e.g. swap local ESO backend for cloud-managed secrets, add Prometheus, wire WIF)

Aim for two to four pages. We value concision and honest reasoning over completeness.

11.1 AI tools disclosure (required)

Your `README.md` **must** include a section with a heading that starts with "AI tools used" (or equivalent: "AI disclosure", "Use of AI"). The shipped harness greps for this heading and fails the run if it is missing. Listing the AI tools you used (Claude, Cursor, Copilot, ChatGPT, etc.), what tasks you used them for, and what you wrote or reviewed independently is what we expect to find under that heading.

We are not penalising AI use — we are penalising lack of transparency. Honest disclosure (including admitting "I had Claude write the entire Kyverno chart, then I read it line by line and adjusted X") is treated the same as no AI use. Submissions without this section will be returned for re-submission.

12. Submission

- Submit a zip of your working tree, including the `.git/` directory.
- `README.md` per §11, including the required **AI tools used** section.
- Paste the output of a successful `make verify` run into the README or a separate `verify-output.txt`.
- `.gitignore` excluding `backend.hcl`, `*.tfvars`, `.terraform/`, `tfplan`, `*.kubeconfig`, `*.tfstate`.

13. Evaluation Criteria

Criteria	What we look for
End-to-end correctness	All 16 checks in <code>make verify</code> pass on a clean clone of your repo.
Terraform module design	Modules have clean interfaces; outputs are consumed; envs/local and envs/local-stage reuse the same modules with different inputs.
Variable validation	At least two <code>validation</code> blocks on critical inputs; sensible regex / range constraints.
Backend convention	<code>backend.hcl</code> gitignored, <code>backend.hcl.example</code> committed; documented init command.
Plan-as-policy	Conftest passes on the JSON plan; you have read the policies and shaped Terraform to match.
GitOps discipline	Root app-of-apps wired correctly; adding a YAML to <code>apps/</code> is enough; sync policy automated; no <code>project: default</code> .
Helm values composition	<code>values.local.yaml</code> and <code>values.local-stage.yaml</code> override only what is necessary; defaults stay in the chart.
Kyverno policy correctness	Policies are enforce-mode and applied via Helm chart deployed by ArgoCD; bad-examples all rejected.
Secrets handling	ESO wired; local <code>ClusterSecretStore</code> ; no plaintext secrets anywhere in repo.
CI pipeline composition	Three workflows, idempotent on PR + push; Conftest gates the plan; Trivy report uploaded.
AI tools disclosure	README contains an "AI tools used" (or equivalent) heading naming the tools, what they did, and what you wrote / reviewed independently. Required — missing section is a fail.
Operational hygiene	Clear README; honest trade-offs.

Good luck. We look forward to reviewing your submission.

— KAD Global Agency, Platform Engineering Team